Vidyalaya ERP — Use-Case Specification

Product	Vidyalaya ERP (multi-school School Management System)
Version	2.0 · 18 October 2026
Audience	School leadership, implementation consultants, QA, and anyone evaluating how the product supports real school operations
Companion documents	User Guide (screen-by-screen instructions with annotated screenshots)

Every use case below is **verified by an automated end-to-end test** that runs the same journey across the real roles. The *Verified by* line names that test. See Appendix A for how to run it.

Contents

ID	Use case	Primary actor	Supporting actors
UC-01	Enquiry to enrolment	Front Office	Principal, Parent
UC-02	Bulk onboarding at go-live	Principal/Admin	Front Office
UC-03	Daily student attendance with absence alerts	Teacher	Parent, Principal
UC-04	Student leave request	Parent	Class teacher, Principal
UC-05	Monthly fee cycle	Accountant	Front Office, Parent
UC-06	Online fee payment	Parent	Accountant
UC-07	Exam cycle to report card	Principal/Admin	Teachers, Parent, Student
UC-08	Homework: assign, submit, review	Teacher	Student, Parent
UC-09	Behaviour logging and parent follow-up	Teacher	Parent
UC-10	Parent–teacher communication	Parent	Teacher
UC-11	Targeted notices	Teacher / Principal	Parents, Students
UC-12	Monthly payroll	Accountant	All staff
UC-13	Library circulation	Librarian	Student
UC-14	Store point of sale	Store Manager	Accountant
UC-15	Transport enrolment and billing	Transport Manager	Parent, Accountant
UC-16	Certificate issue and public verification	Front Office	Principal, any third party
UC-17	Student leaves the school	Front Office	Accountant, Librarian

ID	Use case	Primary actor	Supporting actors
UC-18	Session-end promotion	Principal/Admin	—
UC-19	Government and management reporting	Principal/Admin	Accountant
UC-20	Tailoring roles and permissions	Owner	Principal/Admin
UC-21	Secure staff off-boarding	Principal/Admin	Accountant
UC-22	Onboarding a new school (SaaS operator)	Platform Admin	School Owner
UC-23	Live bus tracking with arrival alerts	Driver	Transport Manager, Parent
UC-24	Hostel admission and billing	Principal/Admin	Accountant, Parent
UC-25	Month-end export to Tally	Accountant	—
UC-26	Install the app and receive phone alerts	Any user	—
UC-27	Choose the school's languages	Owner / Principal	All users
UC-28	Visitor entry and early-leave gate pass	Front Office	Guard, Parent
UC-29	Timed online test with automatic marking	Teacher	Student, Parent
UC-30	PTM poll and trip consent form	Principal/Admin	Parent, Teacher
UC-31	Advance payment and one-off charges	Accountant	Parent
UC-32	Moving from the old software	Principal/Admin, Accountant	—
UC-33	Discipline incident with parent communication	Teacher	Principal/Admin, Parent
UC-34	Compliance documents and renewals	Principal/Admin	Staff, Parents
UC-35	School news on the public website	Teacher, Principal/Admin	Public
UC-36	Alumni network and placement drives	Placement cell	Alumni, Student, Parent

ID	Use case	Primary actor	Supporting actors
UC-37	Integrations: school email, single sign-on, BigBlueButton, accounting	Owner / Principal	—

UC-01 — Enquiry to enrolment

Goal	Turn a walk-in, phone or online enquiry into an enrolled student whose family can sign in.
Primary actor	Front Office
Supporting actors	Principal/Admin (admission decision), Parent
Preconditions	Classes and sections exist for the session. The acting user has *Admissions* create/edit rights and *Students* create rights to admit.
Trigger	A parent walks in, calls, or submits the public form at `/apply/<SCHOOLCODE>`.
Verified by	`workflows.ts` › *Admission: enquiry → pipeline → admit → logins → leaving → delete*

Main flow

1. Front Office records the enquiry (*Admissions → New enquiry*): child's name, class, gender, guardian name and mobile, and source. Online submissions arrive automatically in **Applied**.
2. The system assigns an application number and places the card in the **Enquiry** column.
3. Front Office works the application: logs calls in **Notes**, sets **Next follow-up**, and moves it through **Applied → Docs verified → Test / interview → Seat offered**.
4. The Principal decides to admit. Front Office (or the Principal) clicks **Admit now**, chooses the section, and confirms guardian consent.
5. The system creates the student with an admission number. It links the child to an existing family if the guardian mobile matches, otherwise creates a family. It records DPDP consent, creates student and parent logins, and shows the one-time passwords.
6. Front Office prints the admission letter and login slips.

Alternate flows

- **A1 – Existing family:** in step 5 the child joins the existing family. The parent sees both children under one login.
- **A2 – Applicant withdraws or is not selected:** Front Office clicks **Withdrawn** or **Reject**. The application moves to **Closed** and can be re-opened.

Exception flows

- **E1 – No consent:** admitting without confirming consent is rejected (*400*).
- **E2 – Already admitted:** a second admit attempt is rejected (*409*).
- **E3 – Unauthorised role:** a Teacher trying to change an application is refused (*403*).

Business rules: admission numbers are sequential and gap-free per school; passwords are shown once and must be changed at first sign-in; the guardian's primary mobile is the sibling-linking key.

Postconditions: the student is on class lists, attendance rosters and invoice runs; the application is in **Admitted**; the creation is in the Audit Log.

UC-02 — Bulk onboarding at go-live

Goal	Load an existing school's students and families in one day.
Primary actor	Principal/Admin
Preconditions	Session, classes, sections and subjects are configured.
Verified by	`workflows.ts` › *Bulk import: valid row admitted, bad row reported*

Main flow

1. *Students → Import → Download template*.
2. Fill one row per student (up to 500 rows per file).
3. Upload. The system validates every row, admits valid rows, and lists errors by row number.
4. View and print the generated login credentials.
5. Fix the error rows and upload them again.

Business rules: the same sibling linking, consent and numbering rules as UC-01 apply to every row.

Postconditions: all imported students and families exist with logins; each import is audited.

UC-03 — Daily student attendance with absence alerts

Goal	Record attendance for a section and inform parents of absences within minutes.
Primary actor	Class teacher
Supporting actors	Parent, Principal
Preconditions	The teacher is class teacher of the section, or teaches a subject in it. For SMS, a gateway is configured and absence alerts are on.
Trigger	Start of the school day.
Verified by	`workflows.ts` › *Attendance: class teacher marks, scope + edit window enforced*

Main flow

1. The teacher opens the roster (*Dashboard → Mark now*, or *Attendance → Mark*). Students with approved leave are pre-filled as **Leave**.
2. The teacher clicks **Mark all present**, changes the exceptions (A, L, H), and saves.
3. The system stores the attendance with the teacher's name and time.
4. Parents of students who have *newly* become absent get an in-app alert and an SMS.
5. The Principal's dashboard shows **Absent today**, and the class-wise summary shows the section as marked.
6. Parents see the day on their child's attendance calendar.

Alternate flows

- **A1 – ID-card scanning:** Front Office or a teacher uses **Scan cards** at the gate. Each scan marks the student present.
- **A2 – Correction:** re-saving the same day updates the record and does not re-alert parents who were already alerted.

Exception flows

- **E1 – Another section:** a teacher opening a section they don't teach is refused (*403*).
- **E2 – Edit window passed:** a teacher changing a date older than the school's edit window (default 2 days) is refused.
- **E3 – Another family:** a parent requesting another child's attendance is refused (*403*).

Postconditions: the register, the attendance-eligibility report and report-card attendance are all updated.

UC-04 — Student leave request

Goal	A parent requests leave for a child and the school approves it before the absence.
Primary actor	Parent
Supporting actors	Class teacher or Principal (approver)
Verified by	`workflows.ts` › *Leave: parent applies → teacher scope → admin approves → parent notified*

Main flow

1. The parent opens *Leave → Apply for leave* and enters the type, dates, reason and child.
2. The system checks for overlaps, creates the request as **Pending**, and notifies the class teacher.
3. The class teacher (*Leave → My students*) or an administrator (*To review*) approves it.
4. The parent is notified. On those dates the attendance roster pre-fills the student as **Leave**.

Exception flows

- **E1 – Overlap:** a second application for the same dates is rejected.
- **E2 – Not the class teacher:** a teacher who is not the class teacher cannot approve (*403*).
- **E3 – Self-approval:** nobody can approve their own leave.
- **E4 – Withdrawing after a decision:** only *pending* applications can be withdrawn.

UC-05 — Monthly fee cycle

Goal	Bill every student, collect payments, and follow up on dues.
Primary actor	Accountant
Supporting actors	Front Office (counter), Parent
Preconditions	Fee heads, the fee structure for the session and any concessions are set up.
Verified by	`workflows.ts` › *Fees: dues → part payment → scoped receipt → cancel restores dues*

Main flow

1. On the 1st of the month, the Accountant runs *Fees → Invoices → Generate invoices*. The system creates one invoice per active student from the structure, adds the transport fare and applies concessions.
2. At the counter, Front Office searches the student or parent mobile, optionally turns on **Whole family**, selects invoices, enters the amount and mode (UTR for UPI, cheque details for cheques), and collects.
3. The system issues a sequential receipt, allocates it to the oldest dues first, posts it to Accounts, and notifies the parent.
4. At the end of the day, the Accountant reconciles with *Fees → Report → Today only*.
5. After the due date, the Accountant applies late fees and sends reminders from **Defaulters**.

Alternate flows

- **A1 – Part payment:** invoices become **Partial**, and the balance stays due.
- **A2 – Bounced cheque:** the Accountant cancels the receipt with a reason. The allocation and the ledger entry are reversed, and dues are restored exactly.

Exception flows

- **E1 – Overpayment:** an amount above the total due is rejected.
- **E2 – Digital payment without a reference:** UPI and other digital modes require a UTR.
- **E3 – Unauthorised:** Teachers cannot collect. Front Office can collect but cannot cancel receipts. Families can only open their own receipts.

Business rules: re-running invoice generation for a month skips students who already have an invoice; receipt and invoice numbers are gap-free.

UC-06 — Online fee payment

Goal	A parent pays fees from the phone without visiting the school.
Primary actor	Parent
Preconditions	The school has entered its Razorpay keys and turned online payment on.
Verified by	`workflows.ts` › *Online payment: parent pays in sandbox → receipt → reversal*

Main flow

1. The parent opens *Fees*, selects invoices, and taps **Pay online**.
2. The system creates a gateway order for the exact outstanding amount and records a pending payment.
3. The parent pays by UPI, card, net banking or wallet.
4. The browser returns the gateway signature. The system verifies it with the school's secret, confirms the payment, issues the receipt and notifies the parent.

Alternate flow – A1 Browser closed early: the gateway webhook confirms the payment server-to-server and the receipt is issued anyway.

Exception flow – E1 Tampered or invalid signature: the payment stays pending and the attempt is audited.

UC-07 — Exam cycle to report card

Goal	Plan an exam, collect marks, and publish results and report cards.
Primary actor	Principal/Admin
Supporting actors	Teachers (marks), Parents and Students (results)
Preconditions	Subjects are mapped to classes, teachers are allocated, and the grading scale and pass criteria are set.
Verified by	`workflows.ts` › *Exams: setup → teacher marks (own subject only) → publish → family results*

Main flow

1. The Admin creates the exam (name, term, dates, default maximum marks).
2. The Admin saves the date sheet per class: subjects, dates, maximum and pass marks.
3. Each teacher enters marks for their own subject and section (with **Absent** where needed).
4. The Admin reviews the result sheets (totals, grades, ranks, pass/compartment/fail) and clicks **Publish results**.
5. Parents and students are notified, see the results, and download report cards.

Exception flows

- **E1 – Marks above the maximum** are rejected.
- **E2 – A subject the teacher doesn't teach** is refused (*403*).
- **E3 – Unpublished results** are never visible to families (*403*).
- **E4 – Changes after publishing:** teachers can no longer change marks. Administrators can **Withhold**, correct and re-publish.
- **E5 – Deleting an exam that has marks** is blocked, to protect results.

UC-08 — Homework: assign, submit, review

Goal	Set homework, collect online submissions, and give feedback.
Primary actor	Teacher
Supporting actors	Student, Parent
Verified by	`workflows.ts` › *Homework: post → student submits → teacher reviews → redo/resubmit*

Main flow

1. The teacher posts homework to a section (subject, title, details, due date, optional attachment).
2. Students and parents of that section see it, with notifications.
3. The student submits online (text and/or a photo or PDF).
4. The teacher reviews each submission and marks it **Checked** or **Redo** with feedback.
5. On **Redo**, the student resubmits.

Exception flow – E1: a student outside the section cannot submit (*403*). Teachers can delete only homework they posted.

UC-09 — Behaviour logging and parent follow-up

Goal	Keep a shared, documented record of conduct and co-scholastic skills.
Primary actor	Teacher
Supporting actors	Parent
Verified by	`workflows.ts` › *Behaviour: ratings + shared observation reach the parent*

Main flow

1. The teacher records an observation for a student (Appreciation, General note or Concern) and chooses whether to share it with the parent.
2. Shared observations appear on the parent's view of the child's profile.
3. At term end, the teacher rates behaviour and skill traits from 1 to 5 for the section. The ratings print on the report card.

Exception flows

- **E1 – Student outside the teacher's sections:** refused (*403*).
- **E2 – Retracting:** authors can delete their own observations, for example one shared by mistake. Deleting someone else's needs Delete rights.

UC-10 — Parent–teacher communication

Goal	A parent raises a concern and books a meeting without phone calls.
Primary actor	Parent
Supporting actors	Teacher
Verified by	`workflows.ts` › *Messages & meetings: parent ↔ child's teacher*

Main flow

1. The parent messages the child's teacher in the app. The teacher is notified and replies.
2. The parent requests a meeting (topic, child, mode, time).
3. The teacher confirms it (adding a video link for online meetings), holds it, and marks it **Completed** with an outcome note.

Exception flows

- **E1 – Contact rules:** parents can contact only their children's teachers and the office.
- **E2 – Privacy:** outsiders cannot read a conversation (*403*).
- **E3 – Confirming:** only the host can confirm or complete a meeting.

UC-11 — Targeted notices

Goal	Reach exactly the right audience with an announcement.
Primary actor	Teacher or Principal
Verified by	`workflows.ts` › *Noticeboard: teacher targets own class only*

Main flow: the author writes a notice and chooses the category, roles and classes (optionally pinned, scheduled or expiring, and, for administrators, also sent as SMS). Only the chosen audience sees it and is notified.

Business rules: Teachers can address only students and parents of their own classes, and cannot pin notices or send SMS. Authors can delete their own notices.

UC-12 — Monthly payroll

Goal	Pay staff correctly once a month and give each employee a payslip.
Primary actor	Accountant
Supporting actors	All staff
Verified by	`workflows.ts` › *Payroll: pay → staff sees own slip only → reverse*

Main flow

1. The Accountant opens the salary sheet for the month (calculated from each salary structure) and adjusts bonuses or deductions.
2. The Accountant selects staff, chooses the mode and date, and confirms payment.
3. The system creates sequential payslips, posts the salary expense to Accounts, and notifies each employee.
4. The Accountant downloads the bank transfer sheet.
5. Each employee sees and prints only their own payslip.

Exception flows

- **E1 – Double payment** for the same month is blocked.
- **E2 – Viewing others' slips:** colleagues cannot open another person's payslip (*403*), and non-managers cannot open the salary sheet.
- **E3 – Wrong payment: Reverse payment** removes the slip and its ledger entry.

UC-13 — Library circulation

Goal	Issue and return books with overdue fines.
Primary actor	Librarian
Verified by	`workflows.ts` › *Library: issue → family sees → return with fine rule*

Main flow: the Librarian issues a book to a student or staff member with a due date, and available copies decrease. The student sees it under **My books**. On return, the system calculates the overdue fine (default ₹2 per day, which can be adjusted or waived), posts collected fines to Accounts, and puts the copy back on the shelf.

Exception flow: a Teacher (view-only) cannot issue books.

UC-14 — Store point of sale

Goal	Sell uniforms, books and stationery with GST billing and accurate stock.
Primary actor	Store Manager
Verified by	`workflows.ts` › *Store: sale reduces stock → cancel restores it*

Main flow: the Store Manager scans or picks products, optionally bills to a student, chooses the payment mode and checks out. Stock decreases, the sale posts to Accounts, and a GST bill prints. **Cancel sale** restores the stock.

Exception flow: an Accountant (view-only) cannot bill.

UC-15 — Transport enrolment and billing

Goal	Put a student on a bus route and bill the fare automatically.
Primary actor	Transport Manager
Supporting actors	Parent, Accountant
Verified by	`workflows.ts` › *Transport: assign → family sees route → remove*

Main flow: the Transport Manager assigns the student to a route and stop (pickup, drop or both). The family sees the route, stop, times, vehicle and driver contact. The next invoice run adds the stop's monthly fare automatically. Vehicle insurance, fitness, PUC and permit expiry dates are highlighted before they lapse.

Exception flow: Front Office (view-only) cannot assign students.

UC-16 — Certificate issue and public verification

Goal	Issue tamper-evident certificates that anyone can verify.
Primary actor	Front Office
Supporting actors	Principal (revocation), any third party (verification)
Verified by	`workflows.ts` › *Certificates: issue → public verify → revoke*

Main flow: Front Office chooses a template (TC, Bonafide, Character or Experience) and the recipient, then issues it. The content is frozen and gets a serial number and QR code. Anyone scanning the QR code sees **Genuine certificate** on the public verification page.

Alternate flow – A1 Revocation: an administrator revokes the certificate, and verification then shows **Certificate revoked**.

UC-17 — Student leaves the school

Goal	Close a student's enrolment cleanly while keeping their history.
Primary actor	Front Office
Supporting actors	Accountant (dues), Librarian (books)
Verified by	`workflows.ts` › *Admission…* (status change and sign-in block)

Main flow

1. The Accountant clears any dues, and the Librarian confirms no books are outstanding.
2. Front Office sets the status to **Transferred** (or Passed out or Dropped) with a leaving date and reason.
3. The system removes the student from active lists and invoice runs, and disables the student's login immediately. The parent login is disabled too if no other child is still enrolled.
4. Front Office issues the Transfer Certificate (UC-16).

Business rules: history (attendance, fees, results) is never deleted; deleting a record is allowed only for entries made by mistake that have no fee receipts.

UC-18 — Session-end promotion

Goal	Move every student to the right class for the new session.
Primary actor	Principal/Admin

Main flow: for each class, the Admin lists the students, unticks exceptions, and promotes them to the target class and section. The exceptions are then processed with **Detain**. The final class is processed with **Pass out**. Then the Admin sets up the new session (academic year, fee structure roll-over with % increase, timetable).

Postconditions: each student's class and section reflect the new session, and history is retained.

UC-19 — Government and management reporting

Need	Report	Output
UDISE+ enrolment	Class strength (UDISE+): boys/girls, social category, RTE per section	Print / CSV
Board-exam eligibility	Attendance eligibility below a threshold (default 75%)	Print / CSV
RTE quota list	Student information filtered by RTE, with chosen columns	CSV
PTM directory	Parent contacts by class	Print
Daily cash book	Fee report (Today only) and accounts statement	Print / CSV
Management review	Dashboard KPIs, fee report for the month, income vs expense	Screen / Print

Business rules: every report respects the viewer's data scope; exports are audited; large exports are capped at 5,000 rows and say so when truncated.

UC-20 — Tailoring roles and permissions

Goal	Match the software to the school's own division of duties without code changes.
Primary actor	Owner
Verified by	`rbac-matrix.ts` (869 role × module checks) and the API suite *roles & permissions* section

Main flow: the Owner switches modules on or off for the whole school, and ticks or unticks View / Create / Edit / Delete for each role and module. Changes take effect within about 30 seconds for every user with that role, on both screens and server. **Reset to defaults** restores the product defaults.

Business rules

- The Owner is never restricted.
- Only the Owner can change the Principal/Admin row.
- Parent and Student rights can be narrowed but never widened beyond the defaults.
- Only administrators can hold Roles & Permissions rights.
- Saved overrides can never exceed these ceilings, even if written directly to the database.
- Every change is audited.

UC-21 — Secure staff off-boarding

Goal	Remove a departing employee's access immediately without losing records.
Primary actor	Principal/Admin

Main flow: the Admin marks the employee **Inactive** with a leaving date. The login is disabled and every session is signed out at once. The Admin then reassigns their teacher allocations and class-teacher role, issues an Experience Certificate, and has the final salary settled.

Business rules: any privilege change (deactivation, role change, password reset) invalidates the user's existing sessions; only the Owner can manage administrator accounts.

UC-22 — Onboarding a new school (SaaS operator)

Goal	Bring a new customer school live in minutes.
Primary actor	Platform Admin
Supporting actor	School Owner
Verified by	API suite *platform & subscriptions* section

Main flow

1. The Platform Admin clicks **Onboard school** and enters the school code, name, board, city, plan (trial or paid) and owner details.
2. The system creates the school with a main campus, the current session, grade scale, fee and account heads, certificate templates and an owner login. It shows the owner's one-time password.
3. The owner signs in with the school code, sets a password, and follows the setup order in the User Guide.

Alternate flows

- **A1 – Trial lapses:** the school becomes read-only. Data stays viewable, and changes resume on renewal.
- **A2 – Suspension:** users of a suspended school cannot sign in.

Business rules: modules outside the school's plan cannot be switched on by the school; platform operators can never open school data screens (verified: every school endpoint returns *403* to the platform account).

UC-23 — Live bus tracking with arrival alerts

Goal	Parents know where the school bus is and when it will reach their stop, without calling the driver.
Primary actor	Driver
Supporting actors	Transport Manager (setup), Parent, Student
Preconditions	Live tracking is turned on (School Settings → Languages & app). The vehicle has a linked driver login or a GPS device link. Stops have coordinates.
Trigger	The driver starts the morning pickup.
Verified by	`features.ts` › *Live tracking: driver trip → parent sees bus and ETA → one-time approach alert; GPS device protocol*

Main flow

1. The driver opens **Driver mode** and taps **Start pickup trip**.
2. The phone sends its position at the school's interval (default 15 s), keeping the screen awake and queueing positions when there is no signal.
3. Staff see the bus as **Live** on the Transport live map.
4. The parent opens *Transport* and sees the bus, their stop, the distance and **"Bus is about N min away"**.
5. When the bus comes within the alert distance (default 800 m) of a stop, families at that stop get **"Bus approaching <stop>"** in the app and on their phones. This happens once per stop per trip.
6. At school, the driver taps **End trip**.

Alternate flows

- **A1 – GPS hardware or tracker app:** a device posts to the vehicle's secret link (OsmAnd / Traccar Client protocol), with no phone needed. Positions appear exactly like Driver mode.
- **A2 – Drop trip:** alerts go to students set to *Drop* or *Pickup & drop*.

Exception flows

- **E1 – Tracking off:** trips and positions are refused (*403*).
- **E2 – Not this vehicle's driver:** starting a trip or sending positions is refused (*403*).
- **E3 – A trip is already running:** a second trip on the same vehicle is refused (*409*).
- **E4 – Invalid position:** null-island or out-of-range coordinates are rejected (*400*).
- **E5 – Leaked device link:** rotating it revokes the old link immediately (*404*).

Business rules: families see only their own child's bus, and only if *Show the bus to parents* is on; staff without Transport access and the platform operator cannot see positions; history is deleted after the retention period.

UC-24 — Hostel admission and billing

Goal	Give boarders a bed and bill hostel fees automatically.
Primary actor	Principal/Admin (or a warden granted Hostel rights)
Supporting actors	Accountant, Parent
Verified by	`features.ts` › *Hostel: rooms, allocation rules, family view, fee on invoice, move and vacate*

Main flow

1. The admin adds the hostel (boys, girls or co-ed) and its rooms, with the number of beds and the monthly fee.
2. **Allot room**: picks the student, a room with free beds and a bed label.
3. The family is notified and sees the hostel, room, bed and warden contact.
4. The next monthly invoice run adds "*Hostel (<hostel>, room <n>)*" at the room's fee.

Alternate flows

- **A1 – Move:** the student moves to another room with free beds, and the history is kept.
- **A2 – Leaving: Vacate** frees the bed, and the fee stops from the next invoice.

Exception flows

- **E1 – Student already has a room:** a second active allocation is refused (*409*).
- **E2 – Room full or bed taken:** refused (*409*).
- **E3 – Wrong gender:** boys' and girls' hostels only accept the matching gender (*400*).
- **E4 – Occupied room:** it cannot be deleted (*409*).
- **E5 – Other families:** they cannot see the child's room (*403*), and families cannot list residents (*403*).

UC-25 — Month-end export to Tally

Goal	Move the school's accounts into Tally without re-typing.
Primary actor	Accountant
Verified by	`features.ts` › *Tally export: masters and vouchers, accounts-only*

Main flow

1. *(Once)* Under School Settings → Fees & payments → Tally export, set the Tally company name, the cash and bank ledgers, and any head-to-ledger renames.
2. *(Once) Accounts → Statement → Tally → 1. Ledgers*: import into Tally (*Import → Masters*).
3. *(Each month)* Choose the period, then *Tally → 2. Vouchers for this period*, and import (*Import → Vouchers*).

Business rules: income becomes Receipt vouchers and expenses become Payment vouchers; the money side uses the cash ledger for cash and the bank ledger for UPI, card, cheque and NEFT; debits equal credits in every file (verified); the reference, party and mode go into the narration; every export is audited.

Exception flows: a reversed date range is refused (*400*), and roles without Accounts access are refused (*403*).

UC-26 — Install the app and receive phone alerts

Goal	Parents and staff use Vidyalaya like an app and get alerts on their phone.
Primary actor	Any user
Verified by	`features.ts` › *Phone push: subscription lifecycle and school switch*

Main flow

1. The user opens the school's link and installs it: **Install app** on Android or a computer, or **Share** → **Add to Home Screen** on iPhone.
2. In *My profile* → *App & notifications*, they tap **Turn on notifications** and allow them.
3. Every in-app alert (absence, fee receipt, results, homework, leave decision, message, bus approaching) also arrives on the phone. Tapping it opens the right screen.

Alternate flows

- **A1 – Shared phone:** when another person signs in and turns notifications on, the device moves to them, so alerts never reach the previous user.
- **A2 – Store app:** schools that want a Play Store or App Store listing build the included `mobile/` wrapper. It is the same site, roles and permissions.

Exception flows: if the school has turned phone notifications off, new subscriptions are refused (*403*); non-https push endpoints are rejected (*400*).

UC-27 — Choose the school's languages

Goal	Staff and families use the system in their own language.
Primary actor	Owner or Principal/Admin
Verified by	`features.ts` › *Languages: school chooses, users switch, unknown or disabled codes refused*

Main flow: under School Settings → Languages & app, the admin ticks the languages to offer (any of 12: English, Hindi, Bengali, Marathi, Telugu, Tamil, Gujarati, Kannada, Malayalam, Punjabi, Odia, Assamese) and picks the default. Each user then picks their language from the top bar, and it is remembered on their account.

Exception flows: the default must be one of the enabled languages (*400*); unknown codes are refused (*400*); users cannot pick a language the school hasn't enabled (*400*); only roles with Settings rights can change the list (*403*).

UC-28 — Visitor entry and early-leave gate pass

Goal	Know who is on campus, and release a child during school hours only to an authorised adult, with the parent informed.
Primary actor	Front Office
Supporting actors	Guard (Other Staff), Parent
Verified by	`gap-features.ts` › *Gate: visitor register, gate pass with parent alerts, guard checkout*

Main flow

1. A visitor arrives. Front Office clicks **New visitor**, enters name, mobile, purpose and whom to meet, records the ID type with its last four digits, and clicks **Check in**. A numbered pass (V000123) prints with a QR code.
2. When the visitor leaves, the guard taps **Check out**. **Still inside** always shows who is on campus.
3. A mother comes to take her child to the dentist. Front Office opens **Student gate passes → Issue gate pass**, picks the student and records the reason and who is collecting.
4. The parent is notified straight away (“Gate pass issued … call the school if you did not arrange this”).
5. At the gate the guard taps **Left**; the parent gets “has left school with … at 11:42”.

Exception flows: a full ID number (12 digits) is refused (*400*); a pass cannot be used twice or cancelled after the child left (*409*); teachers can view but not issue passes (*403*); families never see the visitor register (*403*).

UC-29 — Timed online test with automatic marking

Goal	Run a unit test online, mark objective answers instantly and publish results with a review.
Primary actor	Teacher
Supporting actors	Student, Parent
Verified by	`gap-features.ts` › *Online exams: timed attempt, no answer key leak, auto + manual marking, results*

Main flow

1. The teacher clicks **New online exam**, picks class/section, subject, window and duration, optional negative marking, and ticks questions from the question bank. It saves as a draft.
2. **Publish** notifies students and parents.
3. During the window the student clicks **Start exam**. Questions appear in a shuffled order without the answer key; answers save every few seconds; the timer submits automatically at the end.
4. MCQ, true/false and fill-in-the-blank answers are marked at once; the teacher marks written answers under **Answer sheets**.
5. **Publish results**: the student sees the score and a question-by-question review; the parent sees the score.

Exception flows: draft exams cannot be opened (*404*); starting before the window (*400*); a second attempt after submitting (*409*); parents cannot take the exam (*403*); results cannot be published while written answers are unmarked (*409*); timing is locked once anyone has started (*409*); a mark above the question's maximum (*400*).

UC-30 — PTM poll and trip consent form

Goal	Collect parents' choices and consent without paper slips.
Primary actor	Principal/Admin
Supporting actors	Parent, Teacher
Verified by	`gap-features.ts` › *Polls & forms: audience, one vote each, anonymous results, closing*

Main flow

1. The admin creates a **Quick poll** (“Which Saturday suits you for the PTM?”) for parents of selected classes, and a **Form** for trip consent with food preference and emergency contact.
2. Parents in the audience are notified, vote in one tap and see live results; they fill in the form in a dialog.
3. The admin opens **Results**, exports the responses, and **Closes** the form on the last date.

Exception flows: each person answers once unless editing is allowed (*409*); people outside the audience cannot see or answer (*404*); invalid options (*400*); closed forms (*400*). Anonymous forms never show names in results or exports.

UC-31 — Advance payment and one-off charges

Goal	Accept money beyond the dues and use it automatically; bill one-off items outside the fee structure.
Primary actor	Accountant
Supporting actors	Parent
Verified by	`gap-features.ts` › *Advance fees: overpayment kept, used on the next invoice; switchable*

Main flow

1. A parent pays ₹500 more than the dues at the counter. The receipt notes "Advance ₹500 kept for future fees"; the family's Fees page shows the advance.
2. The accountant raises an **Instant invoice** ("Science City trip", ₹300) for the class. The advance pays it immediately and the invoice shows *Paid*.
3. The next monthly run uses whatever advance remains.

Business rules: the advance is never stored as a separate balance — it is always receipts minus allocations, so cancelling a receipt (which reverses everything it paid, including later uses of its advance) can never leave it wrong. Receipt cancellation and advance use lock the same rows in the same order.

Exception flows: with *Accept advance fees* off, paying with no dues is refused (*409*); online gateway payments never create advance; teachers cannot raise invoices (*403*).

UC-32 — Moving from the old software

Goal	Bring opening dues, past receipts, staff and the library catalogue across from spreadsheets.
Primary actor	Principal/Admin, Accountant
Verified by	`gap-features.ts` › *Imports: fee dues/receipts, staff and books — checked before writing*

Main flow: download each template, fill it in Excel, upload, click **Check file**. The system lists every bad row (unknown admission number, impossible date such as 31/02/2026, unknown payment mode, invalid PAN). When every row is valid, **Import** writes them: dues become invoices, receipts are allocated oldest-first like counter collection, staff are added without logins, and books with the same title + author or ISBN add copies.

Exception flows: nothing is written while any row is invalid; future-dated receipts are refused; roles without the module's create right are refused (*403*).

UC-33 — Discipline incident with parent communication

Goal	Keep a fair, consistent record of incidents and involve parents when appropriate.
Primary actor	Teacher
Supporting actors	Principal/Admin, Parent
Verified by	`gap-features.ts` › *Discipline: record, share or hide from family, resolve*

Main flow: the teacher records the incident (category from the school's list, severity, what happened). If shared, the parent is notified and sees it in the app; private staff notes stay internal. The admin records the action taken and marks it **Resolved**.

Exception flows: categories outside the school's list (*400*); students cannot record incidents (*403*).

UC-34 — Compliance documents and renewals

Goal	Never miss a licence renewal; share policies with the right people only.
Primary actor	Principal/Admin
Supporting actors	Teachers, Staff, Parents
Verified by	`gap-features.ts` › *Documents: role-based access, versions and expiry alerts*; *Tasks: assign staff, progress rolls up, comments, access*; *Reminders: personal vs broadcast rights, scheduler sends once*

Main flow

1. The admin uploads the Fire NOC to *Compliance & licences* with a **valid till** date, shared with admin and teachers; the child-protection policy is shared with everyone including parents.
2. Thirty days before expiry a **Renewals due** banner appears. The admin creates a **Task** for the estate manager with a due date, and a **Reminder** for themselves a week before.
3. The new certificate is uploaded with **Edit**; it becomes version 2.

Exception flows: roles a document is not shared with can neither list nor open the file (*403*); only managers upload managed documents (*400*).

UC-35 — School news on the public website

Goal	Publish reviewed school news inside the app and on a public page.
Primary actor	Teacher (author), Principal/Admin (editor)
Verified by	`gap-features.ts` › *News & blog: review before publishing, public news page and cover images*

Main flow: a teacher writes a post with a cover photo and clicks **Send for review**. The editor requests a change; the author is notified, edits and resubmits. The editor publishes it with **Also show on the public news page**. With the page turned on, anyone can read it at `/news/<school code>` without signing in.

Business rules: unpublished posts are invisible to families; authors cannot edit published posts; only covers of public posts are served publicly; the public page shows no internal ids.

UC-36 — Alumni network and placement drives

Goal	Stay connected with former students and help current students into internships.
Primary actor	Principal/Admin (placement cell)
Supporting actors	Alumni, Student, Parent
Verified by	`gap-features.ts` › *Alumni: directory, former students, public self-registration with approval*; *Placements: eligible students apply, shortlist and selection notify the family*

Main flow

1. The admin adds passed-out students to the alumni directory in one step and shares the self-registration link; entries wait for approval.
2. The placement cell posts a drive for eligible classes. Students apply in one tap; staff shortlist and select; the student and parent are notified at each step.

Exception flows: students cannot create drives (*403*); selected students cannot withdraw online (*409*); self-registration needs an email or phone (*400*) and is closed unless the school turns it on (*404*).

UC-37 — Integrations: school email, single sign-on, BigBlueButton, accounting

Goal	Use the school's existing accounts and tools.
Primary actor	Owner / Principal
Verified by	`gap-features.ts` › *Email channel…*, *SSO, App Frame and BigBlueButton settings*, *QuickBooks, Paybooks, data export and the report builder*; unit tests for SSO policy, BBB signing and export formats

Main flow: set up SMTP and send a test email; turn on Google/Microsoft sign-in limited to the school's domain; add the BigBlueButton server so live classes get personal moderator/viewer links; export the month's journal for QuickBooks and the attendance/LOP sheet for Paybooks; add Google Classroom and DIKSHA under **Apps**.

Business rules: passwords and secrets are write-only (never shown again); SSO signs in only existing active users whose verified email matches, never creates accounts; the full data export is owner-only, excludes credentials and is limited to 3 an hour.

Limits (honest): Google/Microsoft sign-in and BigBlueButton need the school's or operator's own OAuth apps / server — they were verified with unit tests and configuration checks, not against live Google, Microsoft or BBB services. Paybooks and QuickBooks are file imports, not live API connections.

Appendix A — Verification

Run these against the demo school with the dev server running:

```
 npm run test:api                       # 114 checks: auth, RBAC samples, tenant isolation,
family scope, settings, platform
npx tsx tests/api/rbac-matrix.ts       # 1170 checks: every role × every module (52 modules) ×
view/create/edit, platform and anonymous lockout
npx tsx tests/api/workflows.ts         # 112 checks: the cross-role journeys above; cleans up
after itself
npx tsx tests/api/features.ts          # 77 checks: live tracking, hostel, languages, phone push,
Tally export
npx tsx tests/api/gap-features.ts      # 216 checks: UC-28 – UC-37 and every other 2026-10
module; ends with a fee ledger invariant check
npx vitest run tests/unit              # 418 unit tests
```

Result on 18 October 2026, against a production build: API 114/114, permissions 1170/1170, new modules 216/216, earlier features 77/77, unit 418/418, a browser smoke test of every new screen for six roles at 375 px and 1280 px (118 visits, no errors or overflow), and every workflow journey except online payment. That flow uses the payment sandbox, which production builds disable by design; it passed against the development server the same day (112/112).

Appendix B — Roles referenced

Owner/Director, Principal/Admin, Teacher, Accountant, Front Office (Receptionist), Librarian, Transport Manager, Store Manager, Other Staff, Parent, Student, Platform Admin. Default rights for each are in the User Guide, Appendix A.